Week 12

SOFTWARE TESTING

Defect testing

Testing programs to establish the presence of system defects

Objectives

To understand testing techniques that are geared to discover program faults

To introduce guidelines for interface testing

To understand specific approaches to object-oriented testing

To understand the principles of CASE tool support for testing

Topics covered

Defect testing

Integration testing

Object-oriented testing

Testing workbenches

The testing process

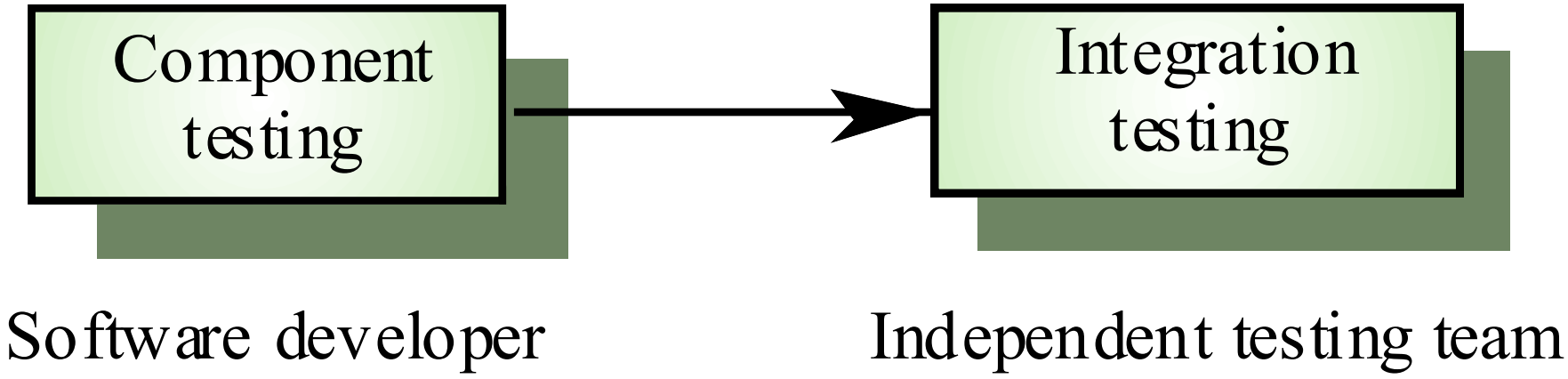
Component testing

- Testing of individual program components
- Usually the responsibility of the component developer (except sometimes for critical systems)
- Tests are derived from the developer's experience

Integration testing

- Testing of groups of components integrated to create a system or sub-system
- The responsibility of an independent testing team
- Tests are based on a system specification

Testing phases



Defect testing

The goal of defect testing is to discover defects in programs

A *successful* defect test is a test which causes a program to behave in an anomalous way

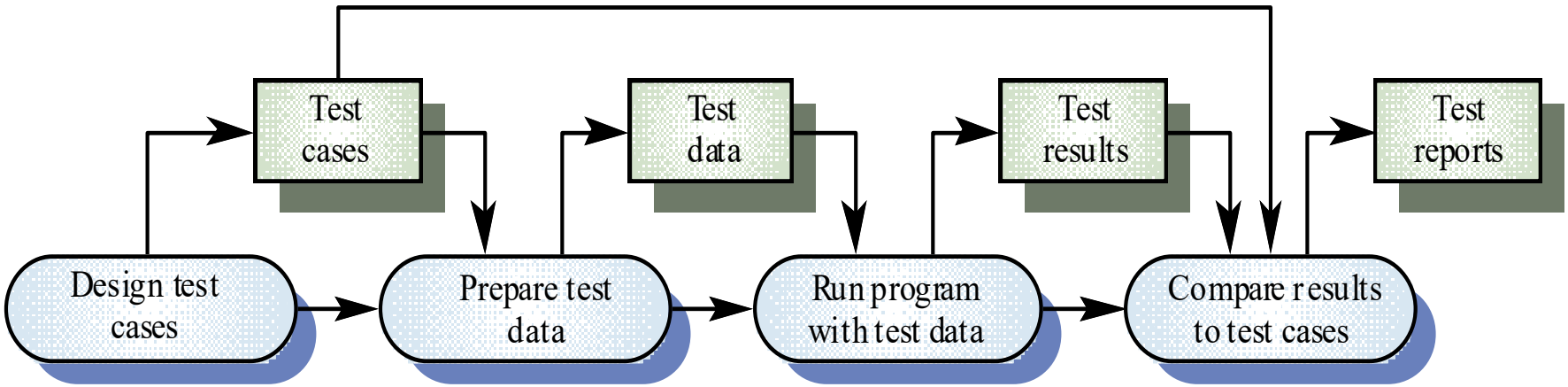
Tests show the presence not the absence of defects

Test data and test cases

Test data Inputs which have been devised to test the system

Test cases Inputs to test the system and the predicted outputs from these inputs if the system operates according to its specification

The defect testing process



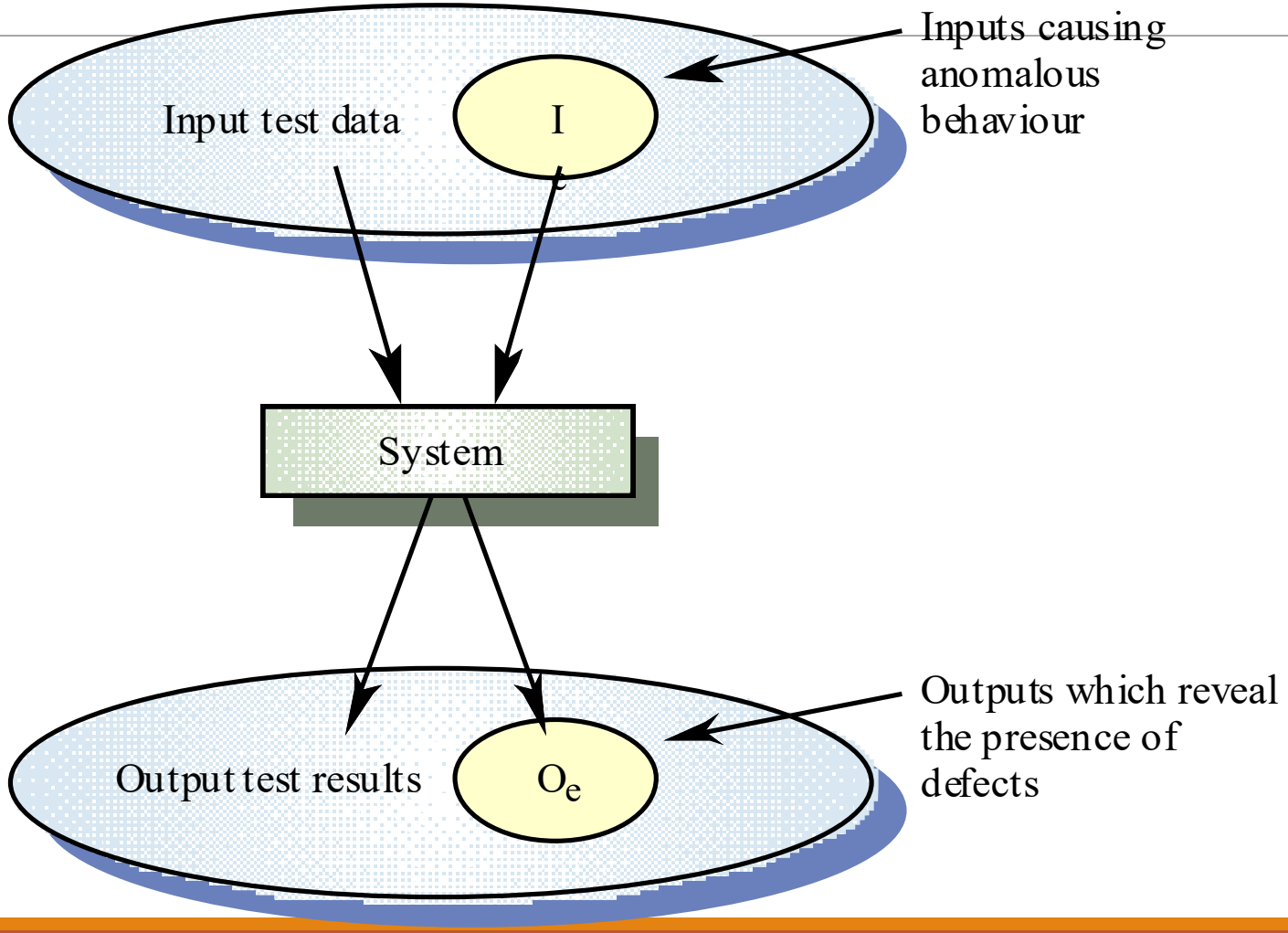
Black-box testing

An approach to testing where the program is considered as a ‘black-box’

The program test cases are based on the system specification

Test planning can begin early in the software process

Black-box testing



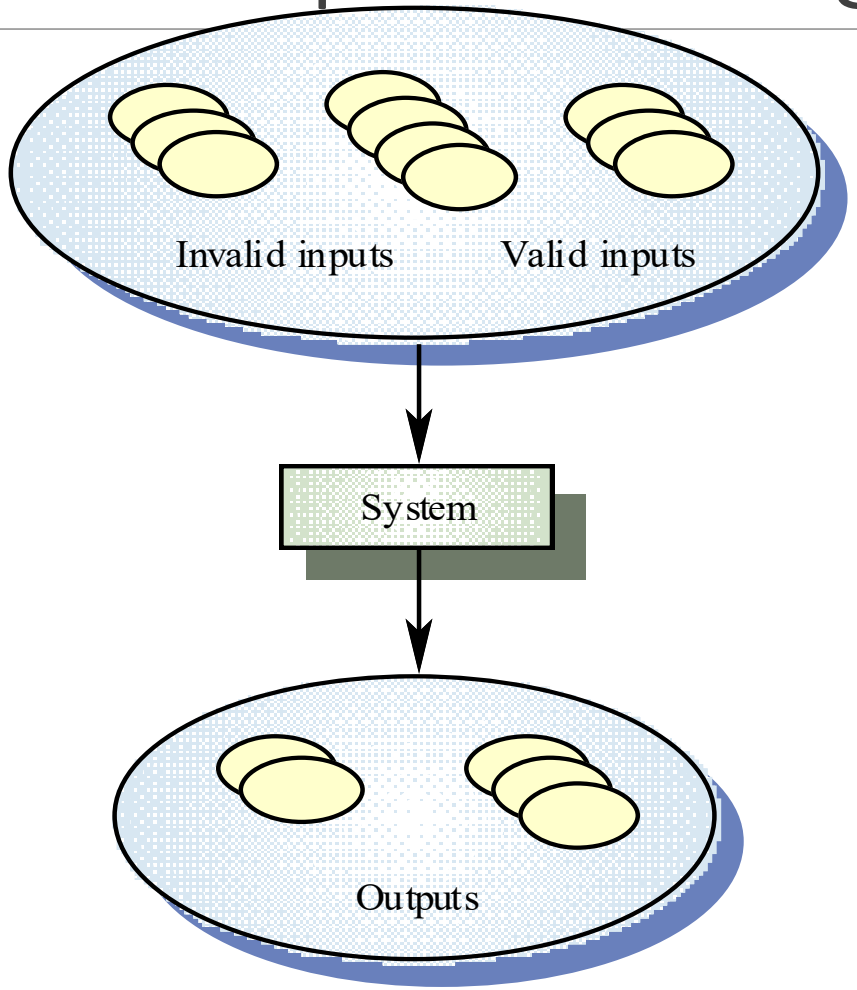
Equivalence partitioning

Input data and output results often fall into different classes where all members of a class are related

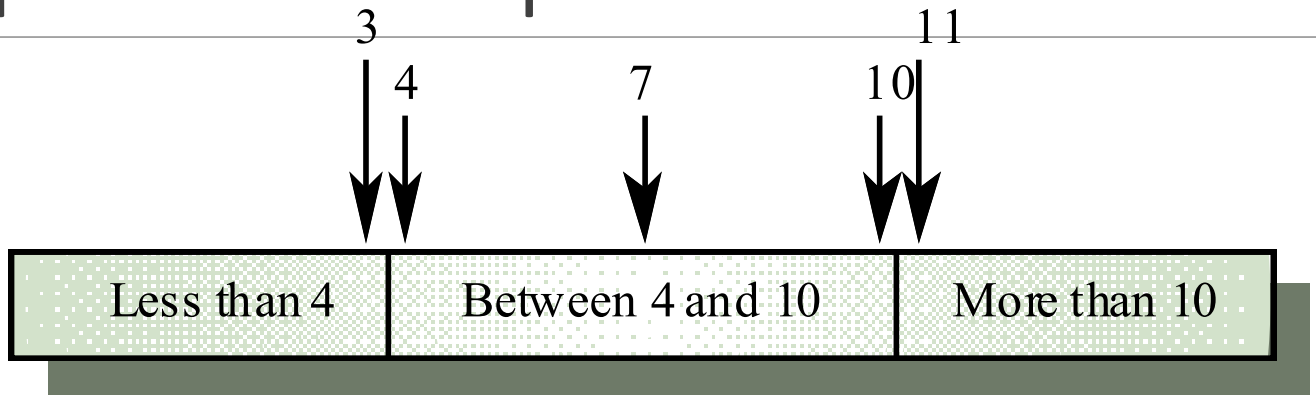
Each of these classes is an equivalence partition where the program behaves in an equivalent way for each class member

Test cases should be chosen from each partition

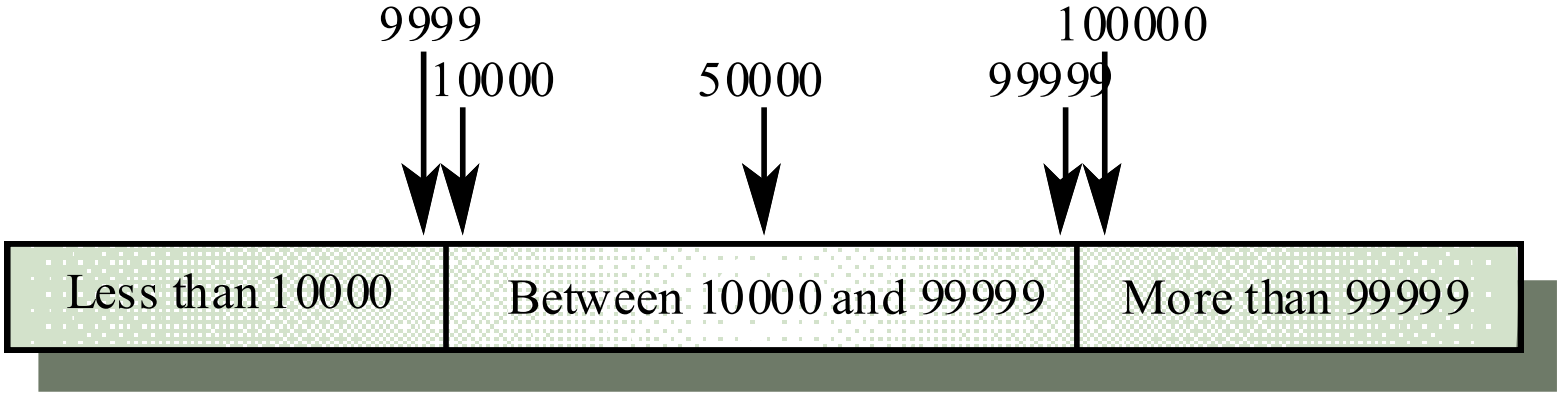
Equivalence partitioning



Equivalence partitions



Number of input values



Input values

Search routine - input partitions

Inputs which conform to the pre-conditions

Inputs where a pre-condition does not hold

Inputs where the key element is a member of the array

Inputs where the key element is not a member of the array

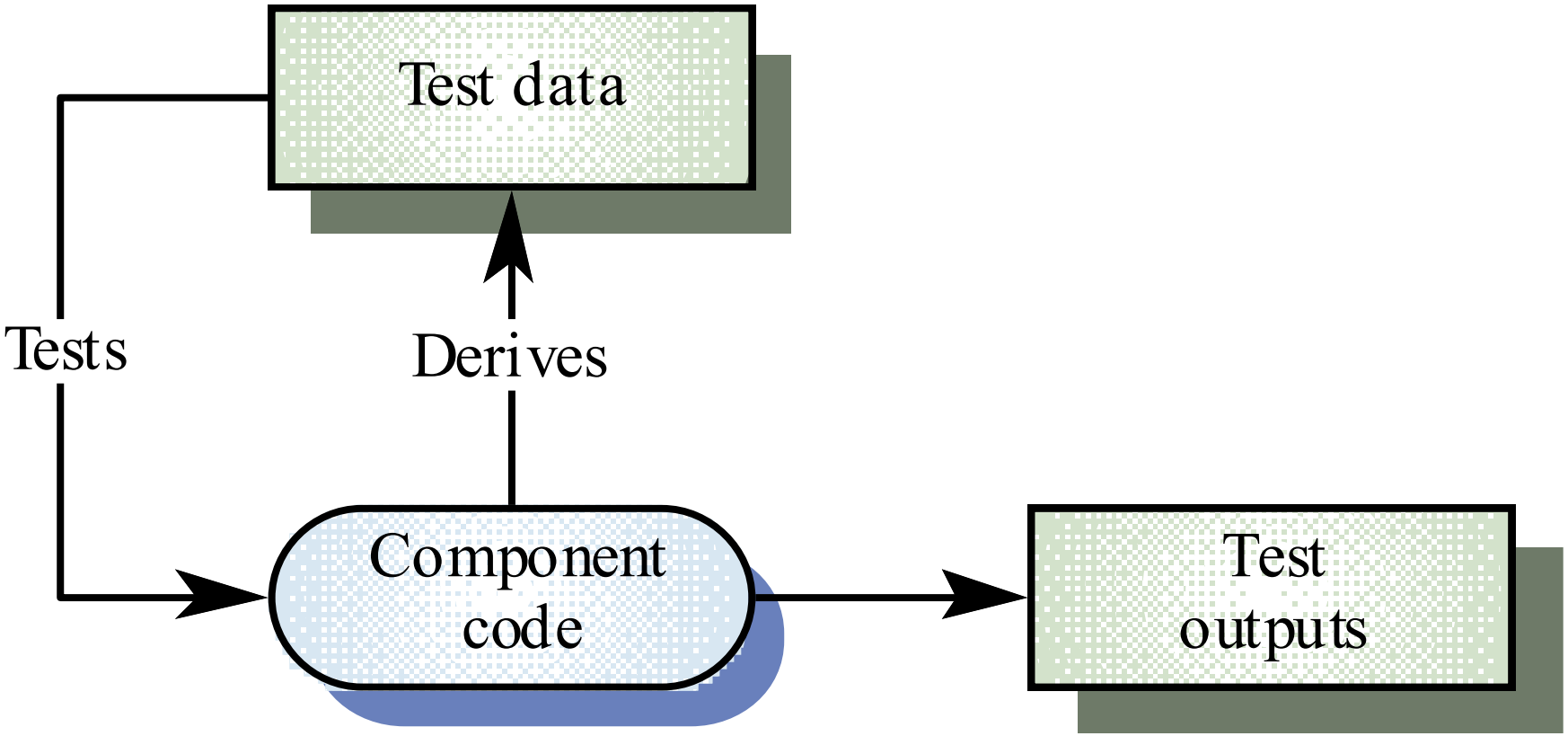
Structural testing

Sometime called white-box testing

Derivation of test cases according to program
structure. Knowledge of the program is used to identify additional test cases

Objective is to exercise all program statements
(not all path combinations)

White-box testing



Path testing

The objective of path testing is to ensure that the set of test cases is such that each path through the program is executed at least once

The starting point for path testing is a program flow graph that shows nodes representing program decisions and arcs representing the flow of control

Statements with conditions are therefore nodes in the flow graph

Integration testing

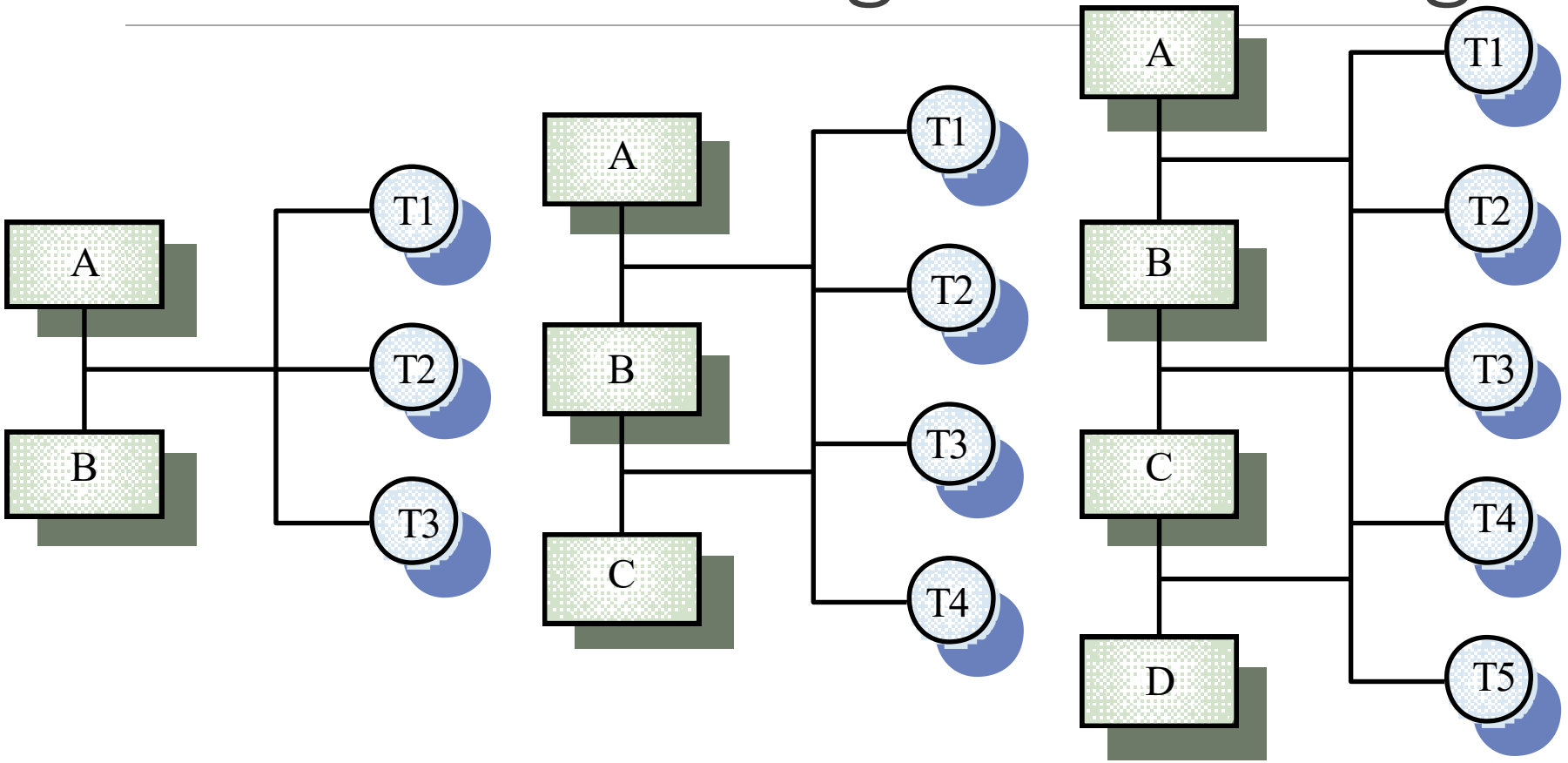
Tests complete systems or subsystems composed of integrated components

Integration testing should be black-box testing with tests derived from the specification

Main difficulty is localising errors

Incremental integration testing reduces this problem

Incremental integration testing



Test sequence 1

Test sequence 2

Test sequence 3

Approaches to integration testing

Top-down testing

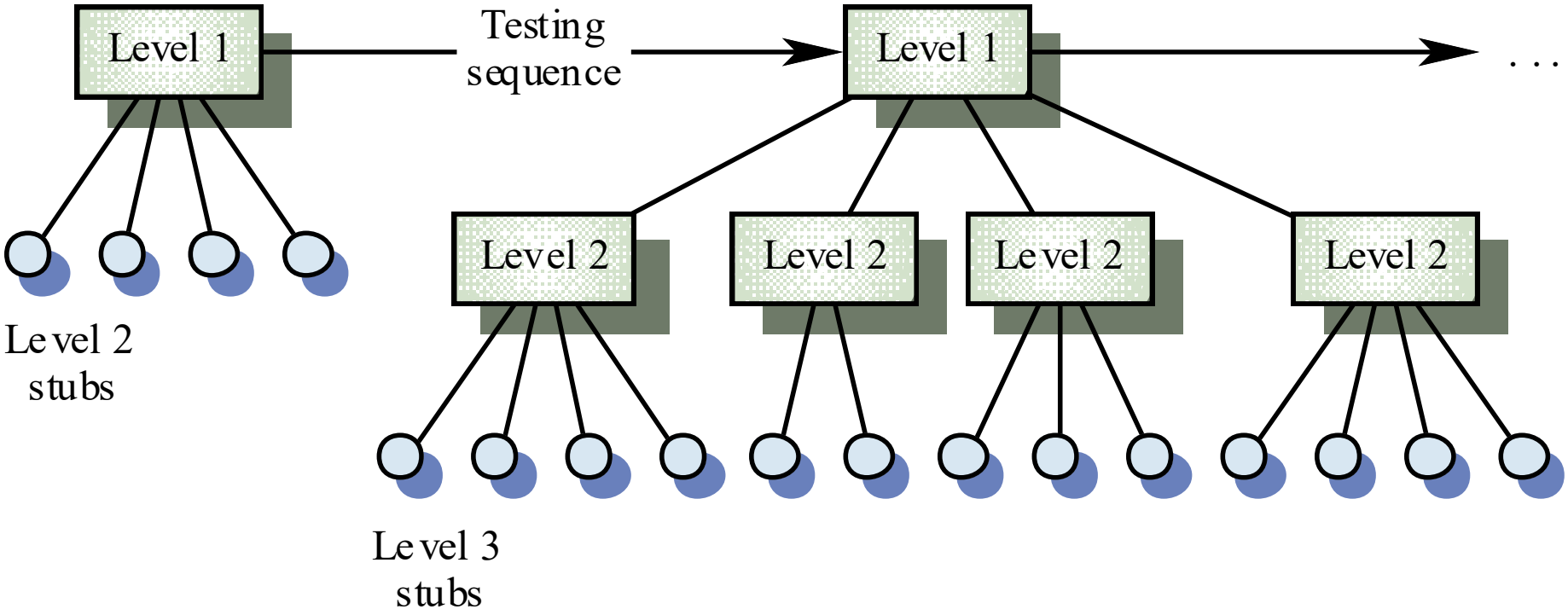
- Start with high-level system and integrate from the top-down replacing individual components by stubs where appropriate

Bottom-up testing

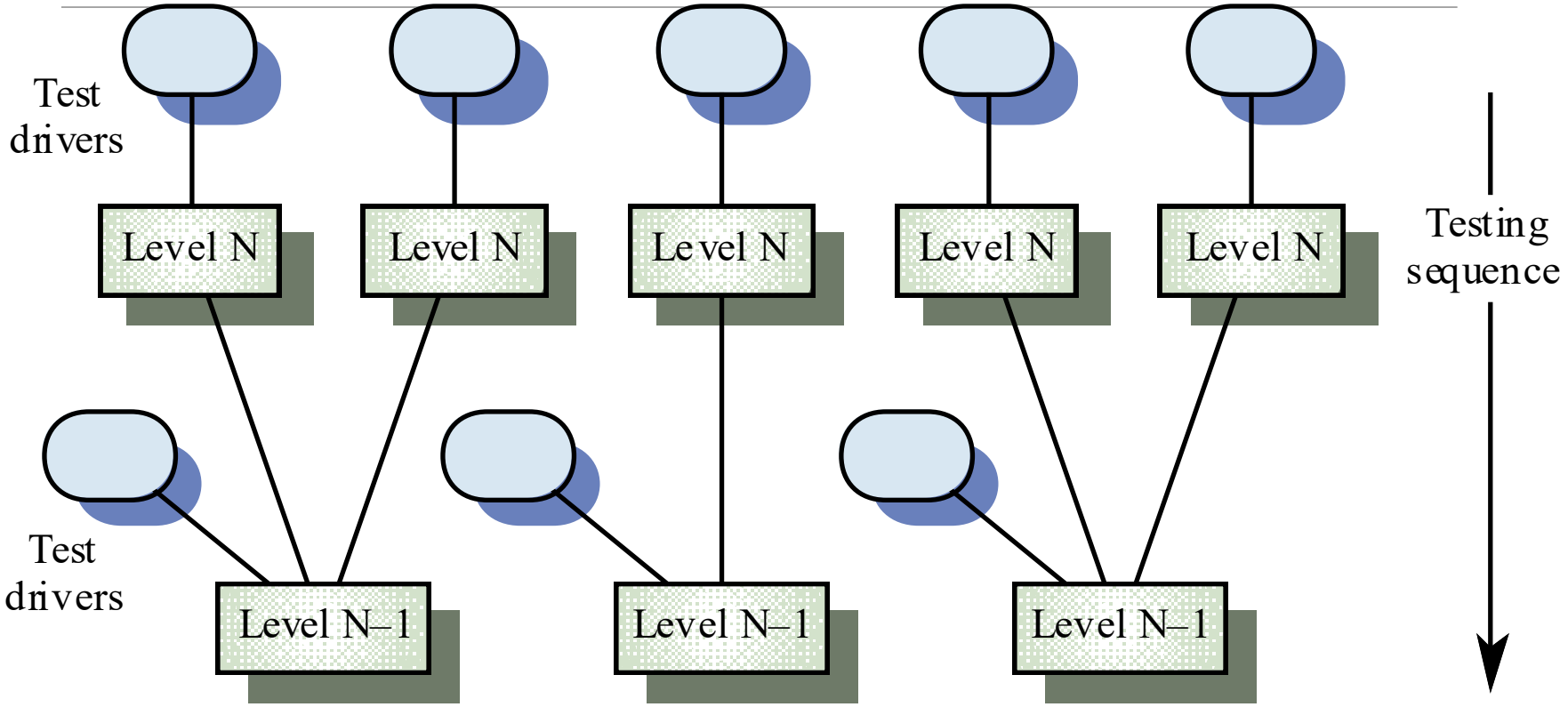
- Integrate individual components in levels until the complete system is created

In practice, most integration involves a combination of these strategies

Top-down testing



Bottom-up testing



Testing approaches

Architectural validation

- Top-down integration testing is better at discovering errors in the system architecture

System demonstration

- Top-down integration testing allows a limited demonstration at an early stage in the development

Test implementation

- Often easier with bottom-up integration testing

Test observation

- Problems with both approaches. Extra code may be required to observe tests

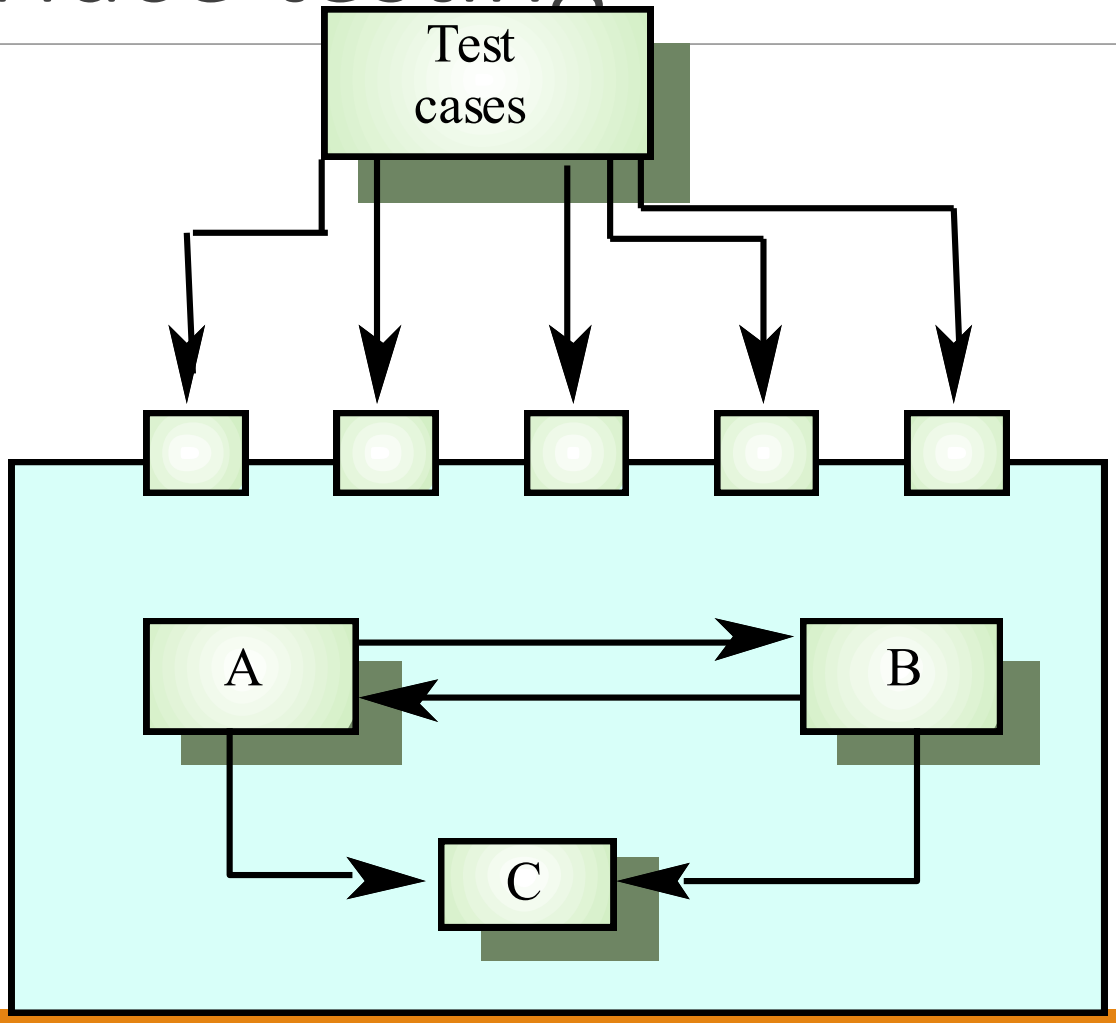
Interface testing

Takes place when modules or sub-systems are integrated to create larger systems

Objectives are to detect faults due to interface errors or invalid assumptions about interfaces

Particularly important for object-oriented development as objects are defined by their interfaces

Interface testing



Interfaces types

Parameter interfaces

- Data passed from one procedure to another

Shared memory interfaces

- Block of memory is shared between procedures

Procedural interfaces

- Sub-system encapsulates a set of procedures to be called by other sub-systems

Message passing interfaces

- Sub-systems request services from other sub-systems

Interface errors

Interface misuse

- A calling component calls another component and makes an error in its use of its interface e.g. parameters in the wrong order

Interface misunderstanding

- A calling component embeds assumptions about the behaviour of the called component which are incorrect

Timing errors

- The called and the calling component operate at different speeds and out-of-date information is accessed

Interface testing guidelines

Design tests so that parameters to a called procedure are at the extreme ends of their ranges

Always test pointer parameters with null pointers

Design tests which cause the component to fail

Use stress testing in message passing systems

In shared memory systems, vary the order in which components are activated

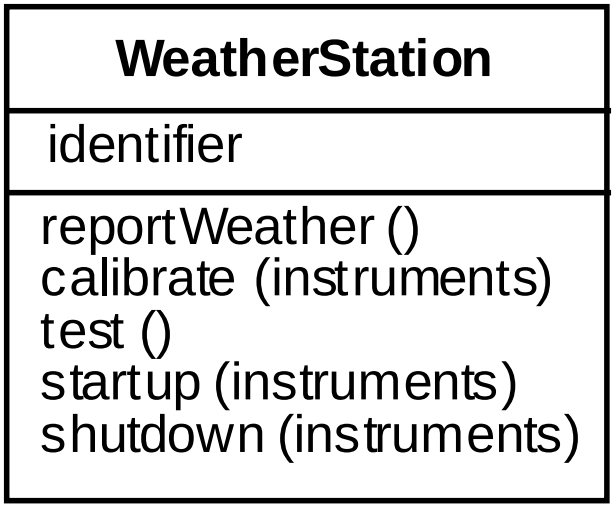
Stress testing

Exercises the system beyond its maximum design load. Stressing the system often causes defects to
come to light

Stressing the system test failure behaviour.. Systems should not fail catastrophically. Stress testing checks for unacceptable loss of service or data

Particularly relevant to distributed systems
which can exhibit severe degradation as a
network becomes overloaded

Weather station object interface



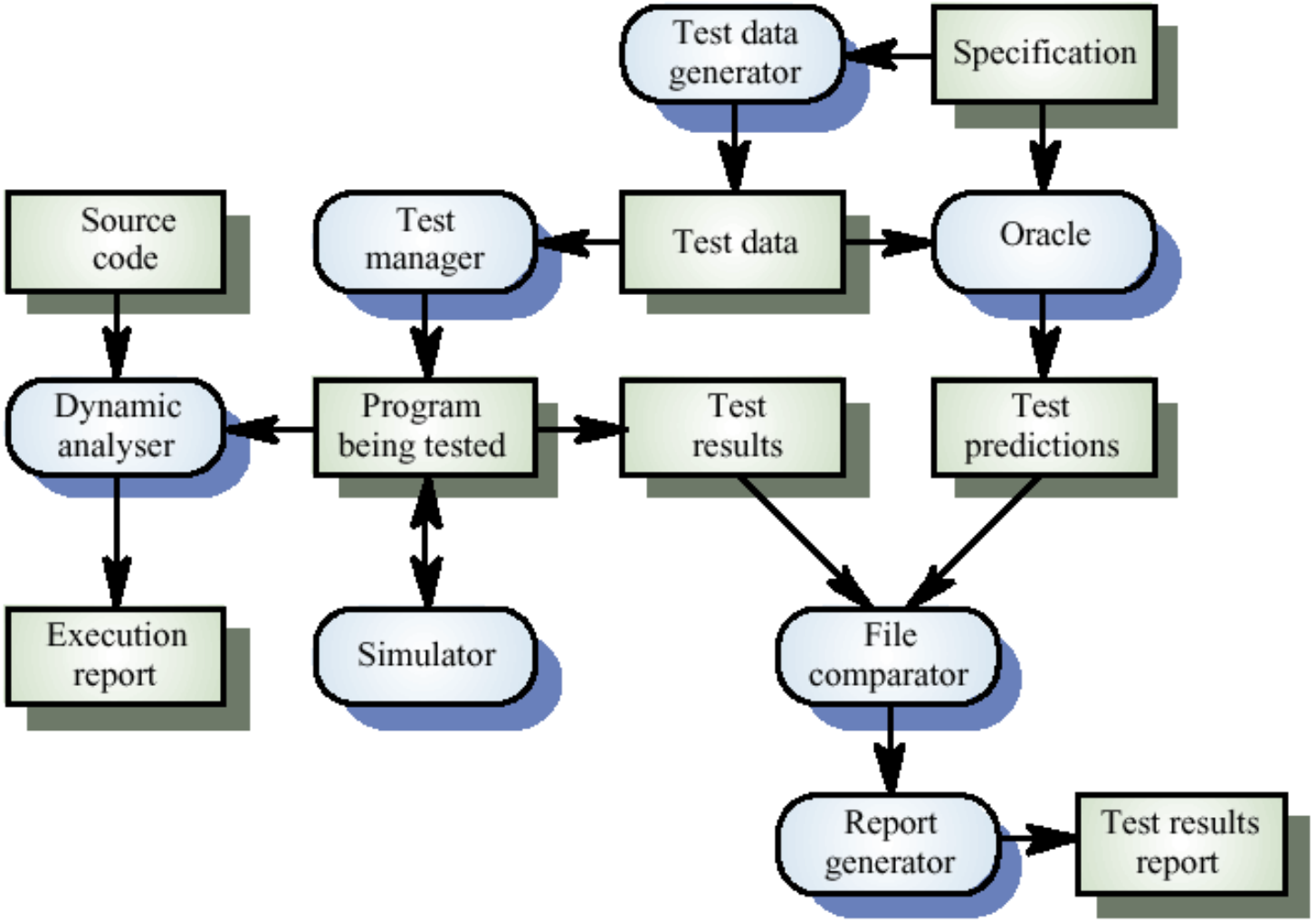
Test cases are needed for all operations

Use a state model to identify state transitions for testing

Examples of testing sequences

- Shutdown → Waiting → Shutdown
- Waiting → Calibrating → Testing → Transmitting → Waiting
- Waiting → Collecting → Waiting → Summarising → Transmitting → Waiting

A testing workbench



Tetsing workbench adaptation

Scripts may be developed for user interface simulators and patterns for test data generators

Test outputs may have to be prepared manually for comparison

Special-purpose file comparators may be developed

Key points

Test parts of a system which are commonly used rather than those which are rarely executed

Equivalence partitions are sets of test cases where the program should behave in an equivalent way

Black-box testing is based on the system specification

Structural testing identifies test cases which cause all paths through the program to be executed

Key points

Test coverage measures ensure that all statements have been executed at least once.

Interface defects arise because of specification misreading, misunderstanding, errors or invalid timing assumptions

To test object classes, test all operations, attributes and states

Integrate object-oriented systems around clusters of objects